

untangle::executor

Generated by Doxygen 1.18.0

1 executor	1
2 Namespace Index	7
2.1 Namespace List	7
3 Class Index	9
3.1 Class List	9
4 File Index	11
4.1 File List	11
5 Namespace Documentation	13
5.1 untangle Namespace Reference	13
5.1.1 Detailed Description	13
6 Class Documentation	15
6.1 untangle::executor< actionT > Class Template Reference	15
6.1.1 Detailed Description	15
6.1.2 Constructor & Destructor Documentation	16
6.1.2.1 executor()	16
6.1.2.2 ~executor()	16
6.1.3 Member Function Documentation	17
6.1.3.1 wait()	17
6.1.3.2 add_task()	17
6.1.3.3 start()	18
6.1.3.4 stop()	18
6.1.4 Member Data Documentation	19
6.1.4.1 on_task_error	19
7 File Documentation	21
7.1 executor.hpp	21
Index	25

Chapter 1

executor

c++ thread pool executor based on async execution

A header-only library: [executor.hpp](#) exposes a pool of workers, each one an [async](#) execution in continuous mode, fed by a shared queue of tasks.

requirements

C++23, and the async submodule:

```
git submodule update --init --recursive
```

async carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>
#include <functional>

int main() {
    untangle::executor<std::function<void(void)>> pool(4);
    pool.start();

    for (int i = 0; i < 100; ++i) {
        pool.add_task([i] { work(i); }, [] { /* finished */ });
    }

    return 0; // ~executor finishes what was added, then stops the workers
}
```

the action type

The pool is a template on the callable it runs, the way an execution is — and on the same parameter name, because every task queued is that one callable. One pool runs one signature, and the caller names it:

```
untangle::executor<std::function<int(void)>> pool(4); // each result goes to the task's callback
```

The action type does not have to be a `std::function`. What an execution requires of it is a nested `result_` type, which it reads rather than deduces, so any callable declaring one will do — and a pool built on such a callable never touches `std::function::result_` type, which the standard removed in C++20 and both `libc++` and `libstdc++` still supply by grace rather than contract.

```
struct my_task {
    using result_type = void;
    void operator()() const;
};

untangle::executor<my_task> pool(4);
```

tasks: work that reports back

Work enters the pool through one door, `add_task()`, and every task carries the callback it must notify. An action type may take arguments, and `add_task()` binds them the way its namesake `execution::add_task()` does — at the door, where the caller still holds them — with the callback after them:

```
untangle::executor<std::function<int(int)>> pool(4);
pool.start();

// the callback comes last, after the arguments the task is bound to
pool.add_task([](int n) { return n * 2; }, 21, [](int result) { /* result == 42 */ });
```

What waits for a worker is one callable carrying its own arguments and its callback, so nothing about a task changes between the queue and the worker that runs it. The arguments are copied at `add_task()` and handed to the task as the pool's own lvalues when it runs: a task is called later, possibly much later, and what the caller passed may be gone by then. A task taking `int&` therefore mutates the pool's copy, which the caller never sees.

A task that cannot be called with the arguments given is refused by a `static_assert` in `add_task()`, which names the mismatch before the errors from inside the binding follow it.

Work that returns `void` still reports that it finished, with a callback taking nothing — finished is the message and the result is optional:

```
untangle::executor<std::function<void(void)>> pool(4);
pool.add_task([] { /* ... */ }, [] { /* finished */ });
```

A task with nothing to bind passes the callback alone, whatever it returns:

```
untangle::executor<std::function<int(void)>> pool(4);
pool.add_task([] { return 42; }, [](int result) { /* result == 42 */ });
```

The callback is the last argument, and the signature cannot say so. A parameter pack cannot be followed by a deducible parameter, so it arrives inside the arguments and `untangle::bind_task()` splits it off. It must be callable with the task's result and return nothing — or callable with nothing at all, when the task returns `void`. A missing or unusable one is a compile error rather than silence.

Work nobody wants told about still passes a callback, one that ignores what it is given. There is no fire-and-forget door: a `void` task's callback costs one empty call, and a single door means a single queue, in which the order out is the order in.

```
constexpr auto ignore_result = [](auto&&...) {}; // takes a result, or nothing for a void task
pool.add_task([](int n) { return n * 2; }, 21, ignore_result);
```

A refused task does not notify. `add_task()` answers false before `start()`, after `stop()`, while a `wait()` is draining, once the destructor has begun, or if the worker it was offered to has stopped — and a refused

task is destroyed rather than queued. The answer is the whole report, so a caller waiting on the callback rather than on the answer would wait for ever. The same is true of work still queued when a stopped pool is destroyed: it never runs, so it never notifies, and the count on stderr is all there is.

Finished does not mean failed. A task that throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to `on_task_error`. And because a callback runs inside the same try as the task that owns it, `on_task_error` can fire for a task that in fact succeeded — the work was done and only the telling failed.

A callback runs on a worker's thread, inside that worker's drain, and several may be running at once on different workers. It is the same rule `on_task_error` carries, for the same reason: nothing may escape it.

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — `on_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `add_task()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

running, stopping, restarting

A pool is built stopped. The constructor makes the workers and wires them up; `start()` runs them. Until it is called the pool refuses work, because a task taken then would sit unrun and the destructor would wait for it. It is also the moment to assign `on_task_error`, before anything can throw.

`stop()` stops the workers, and stops the pool accepting. What a worker is already running it finishes; what is still queued stays queued. `stop()` does not wait for the workers to leave their threads — only the destructor does that, and that wait is what makes destroying the pool safe.

`start()` after `stop()` restarts it, workers and all, and the queue it was holding drains as they pick it up again. Either call is harmless twice over.

`wait()` is a barrier, not a shutdown. It returns once everything added has run and no worker is busy, and it leaves the pool exactly as it found it — workers still in their threads, nothing destroyed, accepting again. Work added after it runs as it did before, and calling it repeatedly is how you step through work in phases:

```
pool.add_task([ { first(); }, [] {}];
pool.wait();
pool.add_task([ { second(); }, [] {}];
pool.wait();
```

// first() has run, the pool is still alive
// and so has second()

It refuses new work while it waits, then restores accepting to whatever it was. A task that re-adds itself would otherwise keep the drain from ever ending, so `add_task()` answers false for the duration — and because the previous state is restored rather than assumed, `wait()` on a stopped pool does not leave it accepting.

So only work accepted before the wait began is what the wait covers. Work offered during it was never queued, and work added after it returns is simply the next batch, for the next `wait()`. This is a stop-the-world drain, not a transparent one: a producer on another thread sees `add_task()` answer false while a `wait()` is in progress, and has to offer the work again afterwards.

A pool that is not running is not waited for. Only a worker takes from the queue, so a stopped pool can never empty it, and a wait insisting otherwise would never return. `wait()` returns at once instead, leaving what is queued for the next `start()`.

lifetime and failure

The destructor finishes what was added. Its drain is `wait()`, so a task added before the pool goes out of scope has run by the time it does. What the destructor adds is the shutdown `wait()` deliberately leaves out, in the order that makes it safe: stop the workers, wait for every one of them to leave its thread, and only then destroy them.

The destructor waits for the pool, not for what the pool's work touched. That distinction is the whole reason `wait()` is public. The drain guarantees no worker is still running when the pool is destroyed, which is a statement about the pool's own members — a destructor body runs before them, so they are all still alive for it. It says nothing about the lifetime of what a task captured.

For a pool that is a local variable outliving what its work touches, the destructor is enough and nothing else is needed. For a pool held as a member of another class, it is already too late: `~executor()` runs during its owner's member destruction, so by then the owner's derived subobject is gone, and so is every member declared after the pool. A callback reaching into either of those runs against destroyed storage however faithfully the pool drains — and on the stack that is not even a fault a sanitizer can name, because the memory is still addressable.

So an owner whose callbacks reach back into itself calls `wait()` where the whole object graph is still standing — in its own destructor body, ahead of its members, or better still at a point the owner of its peers controls — and the destructor's drain goes back to being a backstop rather than the thing relied upon.

A pool destroyed while stopped is the exception: its queue can never empty, because only a running worker takes from it, so the destructor does not wait for it. Whatever is left is reported on `stderr` and dropped.

Both waits are unbounded, and both say so while they last. A task that never returns would otherwise leave the destructor silent for ever, so it reports instead — the first wait naming who is still inside a task, the second naming whose thread has not left. The first line below is the drain's, so a plain `wait()` prints it exactly as a destructor does; the second is the destructor's alone:

```
executor: still waiting to drain after 1s - 1 queued, 2 in a task: pool_worker_0, pool_worker_1
executor: still waiting to drain after 3s - 0 queued, 1 in a task: pool_worker_1
executor: still waiting to stop after 1s - 1 not left its thread: pool_worker_1
```

The interval starts at a second and doubles to a ceiling of thirty, so a stuck teardown says something almost at once without a long one becoming a flood. The two waits are kept apart because they answer different questions: a worker can be idle and still be in its thread.

Bounding either wait was never available. Returning early while a worker is still in a task is a use-after-free, and a bounded wait only moves the hang into `~execution()`, which spins on its own running state with no timeout. Workers are detached and `async::execution` has no cancellation, so a pool cannot outlive a task it cannot interrupt. What can be fixed is the silence.

`add_task()` says whether the task was taken. It returns false before `start()`, after `stop()`, while a `wait()` is draining, and once the destructor has begun. It returns false again if the worker it was offered to has stopped. A refused task is destroyed rather than queued, so a false answer means it will not run.

A task that is still running is refused too, and that is chosen rather than incidental. A pool draining is waiting for exactly that task, so it is tempting to let the work it spawns through — but then a task that re-adds itself could keep the drain from ever ending, and that holds for a `wait()` just as for a destructor. One rule instead: a pool that is not running takes no work, whoever is asking. `add_task()` answers false inside the task, so it can tell.

A task that throws does not take the worker with it, and what it threw is not lost. The worker catches it and carries on with the next task; `on_task_error` is where the throw goes:

```

untangle::executor<std::function<void(void)» pool(4);

pool.on_task_error = [](std::exception_ptr thrown) {
    try {
        std::rethrow_exception(thrown);
    } catch (const std::exception& e) {
        log(e.what());
    }
};

```

It carries a `std::exception_ptr` because a task may throw something that is not a `std::exception`, and that is the case most worth hearing about. Assign it before the first `add_task()` — a worker reads it — and expect it on a worker's thread, with more than one worker possibly inside it at once. Leave it unset and the pool warns on `stderr` instead, naming the worker.

That name is not unique between pools. Workers are named by index — `pool_worker_0`, `pool_worker_1` — so two pools in one process each have a `pool_worker_0`, and every warning that names one is ambiguous: the failure reports above, and the destructor's `wait` alike. With a single pool the name says which worker; with two it does not. Assigning `on_task_error` sidesteps it for failures, since the handler is the pool's own and the warning is replaced.

A return value is not collected by the pool: a continuous worker keeps none. It goes to the task's own callback — see `tasks: work` that reports back above.

building the tests

```

cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build

```

`test/CMakeLists.txt` fetches `googletest` at configure time, so the first configure needs network access.

sanitizers

`EXECUTOR_SANITIZE` builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and `ThreadSanitizer` does not ship for every toolchain.

```

cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan

```

Accepted values are `address`, `thread`, `undefined`, or `empty`. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. `address` and `thread` instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

`tools/make_doc.sh` builds `doc/refman.pdf` from the header and this README. It needs `doxygen`, `tectonic` and `python3`, and it fails on any `doxygen` warning that is not an `obsolete-tag` notice, so an undocumented member or a broken `\ref` does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	13
--------------------------	---	--------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::executor< actionT >	
Runs tasks on a fixed pool of untangle::async::execution workers	15

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp	21
--	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class `executor`
Runs tasks on a fixed pool of `untangle::async::execution` workers.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

N workers in continuous mode behind a shared queue. Nothing overtakes what is already queued.

Chapter 6

Class Documentation

6.1 `untangle::executor< actionT >` Class Template Reference

Runs tasks on a fixed pool of `untangle::async::execution` workers.

```
#include <executor.hpp>
```

Public Member Functions

- `executor` (`std::size_t worker_count`)
Builds `worker_count` workers. They do not run until `start()`.
- `~executor` ()
Drains what was added, then stops the workers and destroys them.
- `void wait` ()
Drains everything added so far, then leaves the pool as it was found.
- `template<typename... Args>`
`bool add_task` (`actionT task`, `Args &&... args`)
Queues work that must report, or hands it to a worker that has nothing to do.
- `std::size_t pending` () `const`
How many tasks are waiting for a worker.
- `void start` ()
Starts every worker, and starts the pool accepting work.
- `void stop` ()
Stops every worker. What they are already running, they finish.

Public Attributes

- `std::function< void(std::exception_ptr)> on_task_error`
Called with whatever the work threw. Assigned by the caller.

6.1.1 Detailed Description

```
template<typename actionT>  
class untangle::executor< actionT >
```

Generated by Doxygen

Runs tasks on a fixed pool of `untangle::async::execution` workers.

`add_task()` is the one door: it carries a callback and reports the result to it.

Template Parameters

actionT	- The callable the pool runs. One pool runs one signature, arguments and result.
---------	--

6.1.2 Constructor & Destructor Documentation

6.1.2.1 executor()

```
template<typename actionT>
untangle::executor< actionT >::executor (
    std::size_t worker_count) [inline], [explicit]
```

Builds worker_count workers. They do not run until [start\(\)](#).

Parameters

worker_count	- How many workers to run. Must not be zero.
--------------	--

Exceptions

std::invalid_argument	- worker_count is zero: such a pool could never run its work.
-----------------------	---

6.1.2.2 ~executor()

```
template<typename actionT>
untangle::executor< actionT >::~~executor () [inline]
```

Drains what was added, then stops the workers and destroys them.

The drain is [wait\(\)](#); the shutdown after it stops the workers, waits for each to leave its thread, and destroys them only then. Both waits are unbounded and report on stderr meanwhile.

Attention

Only a pool stopped before its queue emptied leaves work behind, and work dropped here never runs, so it never notifies. The count on stderr says how much, never which.

This waits for the pool, not for what its work touched: see [wait\(\)](#).

6.1.3 Member Function Documentation

6.1.3.1 wait()

```
template<typename actionT>
void untangle::executor< actionT >::wait () [inline]
```

Drains everything added so far, then leaves the pool as it was found.

Drained means the queue is empty and no worker is busy, so every task accepted before this has run, and so has the callback of one - a callback runs inside the worker's drain, not after it.

A barrier, not a shutdown: the workers keep their threads and work added afterwards runs as before. Reusable, and one at a time - a second caller drains once the first has returned.

The wait is unbounded and reports on stderr while it lasts.

Attention

New work is refused meanwhile, then accepting is restored, so a task that re-adds itself cannot keep the drain from ending. Only work accepted before it began counts.

A pool that is not running is not waited for: only a worker takes from the queue, so this returns at once and what is queued keeps for the next [start\(\)](#).

It waits for the work, not for what the work touched. Held as a member, [~executor\(\)](#) runs once the owner's derived parts and later members are gone: call this before that.

6.1.3.2 add_task()

```
template<typename actionT>
template<typename... Args>
bool untangle::executor< actionT >::add_task (
    actionT task,
    Args &&... args) [inline]
```

Queues work that must report, or hands it to a worker that has nothing to do.

The first free worker takes it, the warmest one; otherwise it waits in the queue, which keeps the order work arrived in. Once the task has run, its callback is invoked with the result, on the worker's thread.

Attention

The last argument is the callback, which the signature cannot say: it arrives in args and [untangle::bind_task\(\)](#) splits it off. A missing or unusable one is a [static_assert](#), and so is a task that cannot be called with the arguments before it.

A task that throws is not notified - what it threw goes to [on_task_error](#) - and a refused one does not notify either, so a caller waiting on the callback alone waits for ever.

An empty callback or action is refused here too. Only a run time check can see it, and a task that can neither run nor notify is turned away rather than queued to throw.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.
------	--

Parameters

task	- The task to run.
args	- What to bind, followed by the callback. Copied here, so a task taking a reference gets the pool's copy.

Returns

true - accepted, and the callback will be notified once it has run.

false - refused by a pool not started, stopped, draining or being destroyed, by a stopped worker, or because the task could never do its job. It will not run or notify.

6.1.3.3 start()

```
template<typename actionT>
void untangle::executor< actionT >::start () [inline]
```

Starts every worker, and starts the pool accepting work.

A pool refuses work until this is called, and again after [stop\(\)](#) until it is called anew, which revives the workers too. Harmless twice over. [wait\(\)](#) needs nothing from here.

6.1.3.4 stop()

```
template<typename actionT>
void untangle::executor< actionT >::stop () [inline]
```

Stops every worker. What they are already running, they finish.

Queued tasks stay unrun and anything added afterwards is refused, a running task included. [start\(\)](#) revives the workers; harmless twice over.

Attention

It does not wait for the workers to leave their threads - only the destructor does, and that wait is what makes destroying the pool safe.

6.1.4 Member Data Documentation

6.1.4.1 on_task_error

```
template<typename actionT>  
std::function<void(std::exception_ptr)> untangle::executor< actionT >::on_task_error
```

Called with whatever the work threw. Assigned by the caller.

The only report of failed work, since [add_task\(\)](#) answered long before it ran. Left unset, the pool warns on stderr. Rethrow the exception_ptr to read it.

Remarks

A task's failure arrives here, and so does its callback's - so this can fire for a task that succeeded, when only the telling failed.

Attention

It runs on a worker's thread, several at once. Assign it before the first [add_task\(\)](#), and let nothing escape it or a callback.

The documentation for this class was generated from the following file:

- executor.hpp

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00008 #pragma once
00009
00010 #include <algorithm>
00011 #include <async.hpp>
00012 #include <atomic>
00013 #include <chrono>
00014 #include <condition_variable>
00015 #include <cstdint>
00016 #include <deque>
00017 #include <exception>
00018 #include <functional>
00019 #include <memory>
00020 #include <mutex>
00021 #include <print>
00022 #include <stdexcept>
00023 #include <string>
00024 #include <thread>
00025 #include <tuple>
00026 #include <type_traits>
00027 #include <utility>
00028 #include <vector>
00029
00030 namespace untangle {
00031
00039 template <typename actionT>
00040 class executor {
00041 public:
00048 explicit executor(std::size_t worker_count) {
00049     if (worker_count == 0) {
00050         throw std::invalid_argument("executor: worker_count must not be zero");
00051     }
00052
00053     workers_.reserve(worker_count);
00054
00055     for (std::size_t index = 0; index < worker_count; ++index) {
00056         auto next = worker_execution::create_instance("pool_worker_" + std::to_string(index));
00057
00058         // Tells the pool the moment this worker has room for more.
00059         next->on_finished = [this, index] { take_next_task(index); };
00060
00061         // Carries what a task throws to the caller. Assigned here because a worker thread reads it.
00062         next->on_error = [this, name = next->name](std::exception_ptr thrown) {
00063             report_task_error(name, thrown);
00064         };
00065
00066         // Adds the worker to the poll the destructor waits on.
00067         poll_.add(*next);
00068
00069         workers_.push_back(std::move(next));
00070     }
00071 }
00072
00084 ~executor() {
00085     // Nothing is stopped before this, or the queue could never empty. wait() leaves the pool
00086     // accepting, so stop() follows at once: that window is the only one a shutdown would take.
```

```

00087     wait();
00088     stop();
00089
00090     {
00091         std::lock_guard<std::mutex> lock(mutex_);
00092
00093         // Only reachable for a pool that was already stopped. wait() drained a running one, and
00094         // stop() above cannot add to the queue.
00095         if (!pending_.empty()) {
00096             std::println(stderr,
00097                 "executor: {} queued task(s) dropped, the pool was stopped before they ran",
00098                 pending_.size());
00099         }
00100     }
00101
00102     // Waits for every worker to leave its thread, which is what makes destroying them safe. A
00103     // different question from the wait above: a worker can be idle and still be in its thread.
00104     auto interval = std::chrono::milliseconds(report_first_ms);
00105     auto waited = std::chrono::milliseconds(0);
00106
00107     while (poll_.is_running()) { // time of check
00108         const auto tick = std::chrono::milliseconds(poll_interval_ms);
00109         std::this_thread::sleep_for(tick);
00110         waited += tick;
00111
00112         if (waited >= interval) {
00113             std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00114                 waited.count() / 1000, count_workers(&worker_execution::is_running),
00115                 name_workers(&worker_execution::is_running));
00116
00117             interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00118         }
00119     }
00120
00121     // time of use: no worker thread is left to reach workers_.
00122     workers_.clear();
00123 }
00124
00145 void wait() {
00146     // One drain at a time: two threads between the exchange below and the restore at the end would
00147     // read each other's cleared flag, and the last one out could leave the pool refusing work.
00148     std::lock_guard<std::mutex> drain(wait_mutex_);
00149
00150     // Put back exactly as found below: a wait must not leave a stopped pool accepting, and such a
00151     // pool cannot empty its queue anyway - only a running worker takes from it.
00152     const bool was_running = running_.exchange(false);
00153
00154     {
00155         std::unique_lock<std::mutex> lock(mutex_);
00156
00157         // Waits for the queue to empty and every worker to go idle.
00158         auto interval = std::chrono::milliseconds(report_first_ms);
00159         auto waited = std::chrono::milliseconds(0);
00160
00161         while (!finished_cv_.wait_for(lock, interval, [this, was_running] {
00162             return nothing_running() && (pending_.empty() || !was_running);
00163         })) {
00164             waited += interval;
00165
00166             // Under the lock, so the counts agree with the predicate that just failed.
00167             std::println(
00168                 stderr, "executor: still waiting to drain after {}s - {} queued, {} in a task{}",
00169                 waited.count() / 1000, pending_.size(), count_workers(&worker_execution::is_busy),
00170                 name_workers(&worker_execution::is_busy));
00171
00172             interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00173         }
00174     }
00175
00176     running_ = was_running;
00177 }
00178
00205 template <typename... Args>
00206 bool add_task(actionT task, Args&&... args) {
00207     // The call the pool will make, which is not quite the one written at the call site: what
00208     // reaches the task is the copies bound below, so that is the call that has to compile. Without
00209     // this the mismatch surfaces deep inside bind_task()'s std::apply.
00210     static_assert(invocable_with_bound<Args...>(),
00211         "executor: the task cannot be called with the arguments given to add_task() "
00212         "before its callback");
00213
00214     if (!running_) {
00215         return false;
00216     }
00217
00218     // untangle::bind_task() takes the callback off the end of the pack and checks it; what comes
00219     // back is one callable that runs the work and then notifies, which is what the queue holds.

```

```

00220     auto tsk = untangle::bind_task(std::move(task), std::forward<Args>(args)...);
00221
00222     // bind_task() builds an empty one from an empty action or callback. Queued, it would throw on a
00223     // worker and report to on_task_error rather than to the caller, who could still act on it.
00224     if (!tsk) {
00225         return false;
00226     }
00227
00228     return queue_task(std::move(tsk));
00229 }
00230
00231 std::size_t pending() const {
00232     std::lock_guard<std::mutex> lock(mutex_);
00233     return pending_.size();
00234 }
00235
00236 void start() {
00237     // The workers first, so anything accepted after this is handed to one that is running.
00238     for (auto& one : workers_) {
00239         one->start();
00240     }
00241
00242     running_ = true;
00243 }
00244
00245 void stop() {
00246     // First, so that nothing is accepted for workers that are about to stop. A task already on a
00247     // worker still runs; one arriving from here is refused.
00248     running_ = false;
00249
00250     // No lock: a worker waking from stop() runs on_finished, which takes mutex_.
00251     for (auto& one : workers_) {
00252         one->stop();
00253     }
00254 }
00255
00256 std::function<void(std::exception_ptr)> on_task_error;
00257
00258 private:
00259 void report_task_error(const std::string& worker, std::exception_ptr thrown) {
00260     if (on_task_error) {
00261         on_task_error(thrown);
00262         return;
00263     }
00264
00265     try {
00266         std::rethrow_exception(thrown);
00267     } catch (const std::exception& e) {
00268         std::println(stderr, "warning: executor task on '{}' threw: {}", worker, e.what());
00269     } catch (...) {
00270         std::println(stderr, "warning: executor task on '{}' threw an unknown type", worker);
00271     }
00272 }
00273
00274 template <typename... Args>
00275 static constexpr bool invocable_with_bound() {
00276     if constexpr (sizeof...(Args) == 0) {
00277         return true;
00278     } else {
00279         return invocable_with<std::tuple<std::decay_t<Args>...>>(
00280             std::make_index_sequence<sizeof...(Args) - 1>{});
00281     }
00282 }
00283
00284 template <typename tupleT, std::size_t... index>
00285 static constexpr bool invocable_with(std::index_sequence<index...>) {
00286     return std::is_invocable_v<actionT, std::tuple_element_t<index, tupleT>&...>;
00287 }
00288
00289 using worker_execution = untangle::async::execution<untangle::task_t>;
00290
00291 bool queue_task(untangle::task_t task) {
00292     std::lock_guard<std::mutex> lock(mutex_);
00293
00294     // The queue comes first: an entry does not overtake one already waiting in it.
00295     if (pending_.empty()) {
00296         // Always from the start, so a free pool reuses its warmest worker rather than waking a cold
00297         // one. Under load the workers are busy and the queue below is what spreads the work.
00298         for (std::size_t index = 0; index < workers_.size(); ++index) {
00299             if (!workers_[index]->is_busy()) {
00300                 // A refused entry is already destroyed, and stop() stops every worker together, so there
00301                 // is nothing to queue and no other worker to try.
00302                 return give_to_worker(index, std::move(task));
00303             }
00304         }
00305     }
00306 }
00307
00308 }
00309
00310 }

```

```

00360     pending_.push_back(std::move(task));
00361     return true;
00362 }
00363
00370 [[nodiscard]] bool give_to_worker(std::size_t index, untangle::task_t task) {
00371     // The door takes the worker's action_mutex under mutex_. The two never cycle, because a worker
00372     // calls on_finished with its action_mutex released.
00373     return workers_[index]->add_action(std::move(task));
00374 }
00375
00379 void take_next_task(std::size_t index) {
00380     std::lock_guard<std::mutex> lock(mutex_);
00381
00382     if (!pending_.empty()) {
00383         untangle::task_t next = std::move(pending_.front());
00384         pending_.pop_front();
00385
00386         if (!give_to_worker(index, std::move(next))) {
00387             // The task is already destroyed, so it is reported rather than put back.
00388             std::println(stderr, "executor: {} refused a queued task, which is lost",
00389                 workers_[index]->name);
00390         }
00391     }
00392     return;
00393 }
00394
00395 if (nothing_running()) {
00396     finished_cv_.notify_all();
00397 }
00398 }
00399
00405 bool nothing_running() const {
00406     for (const auto& one : workers_) {
00407         if (one->is_busy()) {
00408             return false;
00409         }
00410     }
00411     return true;
00412 }
00413 }
00414
00420 std::size_t count_workers(bool (worker_execution::*state)() const) const {
00421     std::size_t count = 0;
00422
00423     for (const auto& one : workers_) {
00424         if ((one.get()->*state)()) {
00425             ++count;
00426         }
00427     }
00428     return count;
00429 }
00430 }
00431
00438 std::string name_workers(bool (worker_execution::*state)() const) const {
00439     std::string names;
00440
00441     for (const auto& one : workers_) {
00442         if ((one.get()->*state)()) {
00443             names += names.empty() ? " : " : ", ";
00444             names += one->name;
00445         }
00446     }
00447     return names;
00448 }
00449 }
00450
00452 static constexpr int poll_interval_ms = 50;
00453
00455 static constexpr int report_first_ms = 1000;
00456 static constexpr int report_max_ms = 30000;
00457
00458 mutable std::mutex mutex_;
00459 std::condition_variable finished_cv_;
00460
00462 std::mutex wait_mutex_;
00463
00466 async::execution_poll poll_;
00467
00469 std::deque<untangle::task_t> pending_;
00470 std::vector<std::shared_ptr<worker_execution>> workers_;
00473 std::atomic_bool running_ = {false};
00474 };
00475
00476 } // namespace untangle

```

Index

- ~executor
 - untangle::executor< actionT >, [16](#)
- add_task
 - untangle::executor< actionT >, [17](#)
- executor, [1](#)
 - untangle::executor< actionT >, [16](#)
- on_task_error
 - untangle::executor< actionT >, [19](#)
- start
 - untangle::executor< actionT >, [18](#)
- stop
 - untangle::executor< actionT >, [18](#)
- untangle, [13](#)
- untangle::executor< actionT >, [15](#)
 - ~executor, [16](#)
 - add_task, [17](#)
 - executor, [16](#)
 - on_task_error, [19](#)
 - start, [18](#)
 - stop, [18](#)
 - wait, [17](#)
- wait
 - untangle::executor< actionT >, [17](#)